

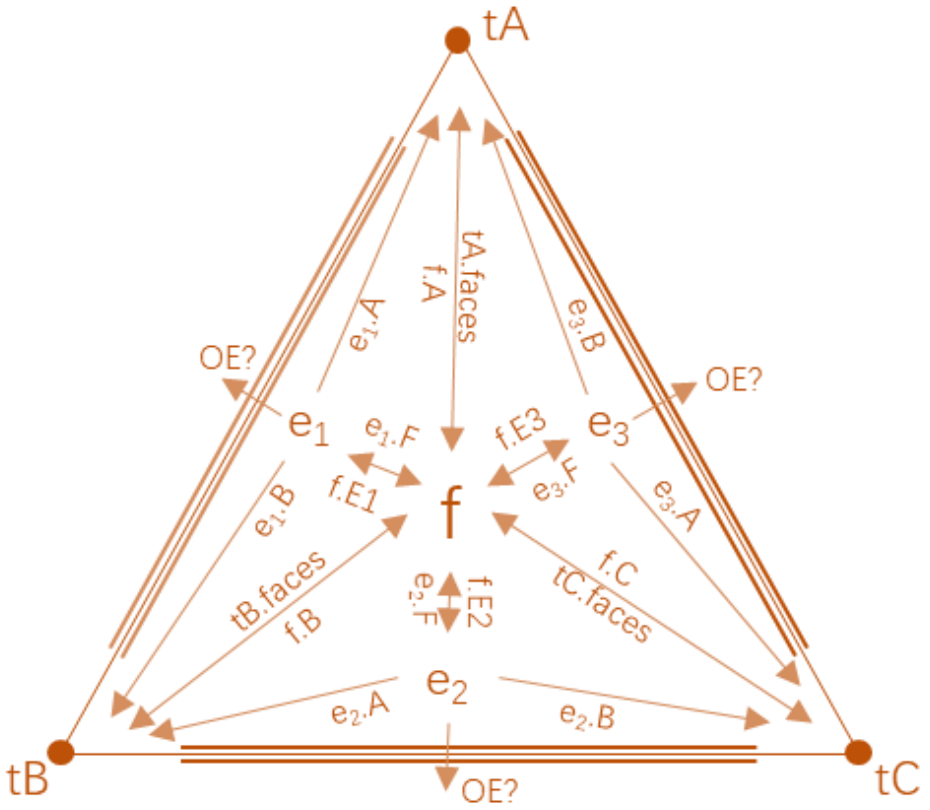
计算机辅助制造实践

——Python实现三维打印路径规划

浙江大学机械工程学院 林志伟
玉泉教1-215　zjtzhylin1986@163.com

第7章 基于拓扑模型的切片轮廓计算

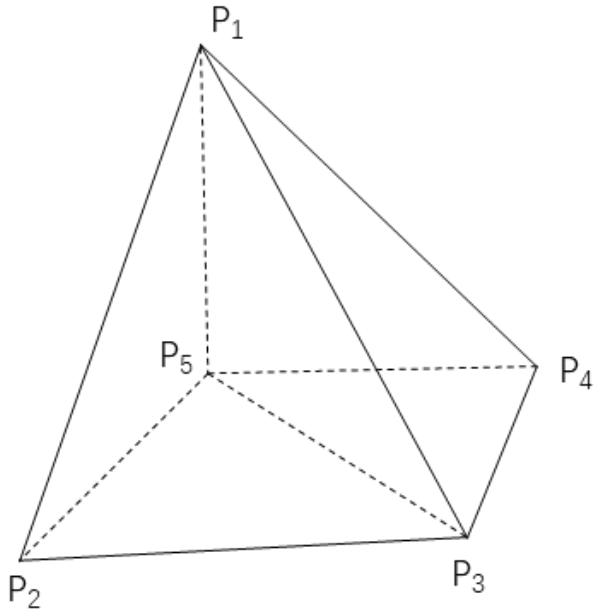
- STL模型存在的问题
- 拓扑关系表示
- STL拓扑模型重建
- 拓扑模型切片
- 算法测试比较



1. STL模型存在的问题

- **问题1. 面片无序保存**
  - STL文件保存的是以三角面片为单元、面片顶点坐标为基础的非常原始的几何数据，其中两点组成一边，三点组成一面，且所有面片是无序保存的，这给切片和截交线段拼接带来了不少麻烦。
- **问题2. 顶点冗余保存**
  - 在STL文件中，各个面片中的各个顶点都是被独立保存的，这直接导致了STL模型的顶点冗余问题。

- 图中的四棱锥拥有**5个**顶点，理论上，用5个独立顶点就足以表示这个模型；但如果用STL模型来表示这个四棱锥的话，那么至少需要6个三角面片，其中每个三角面片包含3个独立顶点，共需**18个**独立顶点，每个坐标位置至少有3~4个顶点重合。
- 信息量没有丝毫增加，顶点数却增加了若干倍，这直接导致了STL模型在保存时占用磁盘或内存空间的增长，这也正是STL模型顶点冗余的最大问题。



1. STL模型存在的问题

- **STL顶点冗余数理论分析**

由**欧拉定理**可知，对任意简单多面体，它的定点数 **V**、面片数 **F** 和棱边数 **E** 满足：

$$V + F - E = 2. \tag{7-1}$$

对STL模型，每个面都是三角形，因此，**每个面包含3条边，而每条边又被2个面共用**，因此有：

$$E = \frac{3}{2}F. \tag{7-2}$$

式(7-2)理解：不妨将一条边拆成两个**半边**，其中一个半边属于一个三角形，另外一个半边属于另一个三角形，这样半边总数必然是3F；然后再考虑将两个半边合并成一条边，这样就可以得到总边数为1.5F。

记STL模型中实际的顶点数为V_{act}，那么显然：

$$V_{act} = 3F. \tag{7-3}$$

根据式(7-1)和(7-2)可得：

$$V = 2 + \frac{1}{2}F. \tag{7-4}$$

式(7-4)中的 **V 表示理论所需的顶点数。**进而可得：

$$\frac{V_{act}}{V} = \frac{6F}{4+F}. \tag{7-5}$$

1. STL模型存在的问题

- **STL顶点冗余数理论分析**

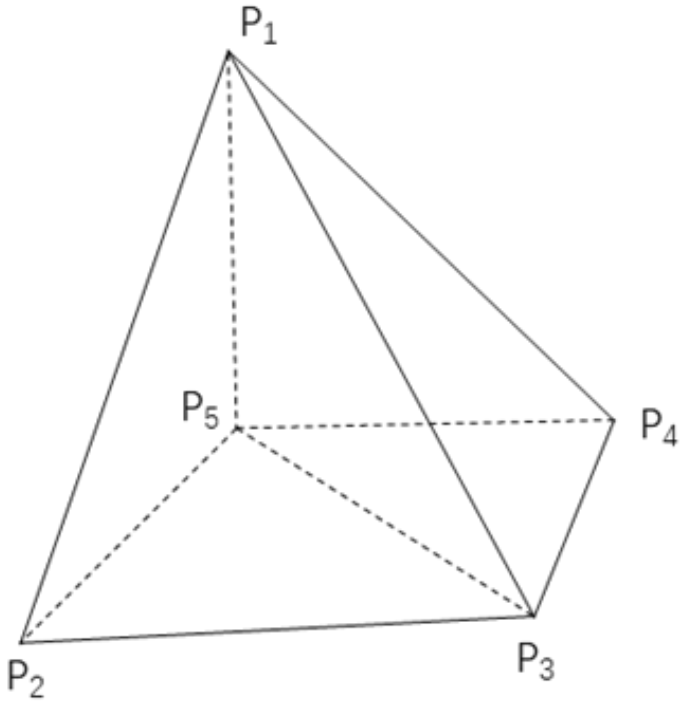
$$\frac{V_{act}}{V} = \frac{6F}{4+F}. \qquad (7\text{-}5)$$

根据式（7-5）：

- 当F = 6时，$V_{act} / V = 3.6$，这就是图中模型的情况。
- 对一般情况，当面片数 **F 远大于 4**，此时分母中的常数 4 可以忽略不计，那么STL模型中**实际顶点数是理论上所需顶点数的6倍**。

这意味着什么?

对 10MB 的模型数据，如果保存成STL格式，可能需要 60MB 的容量。



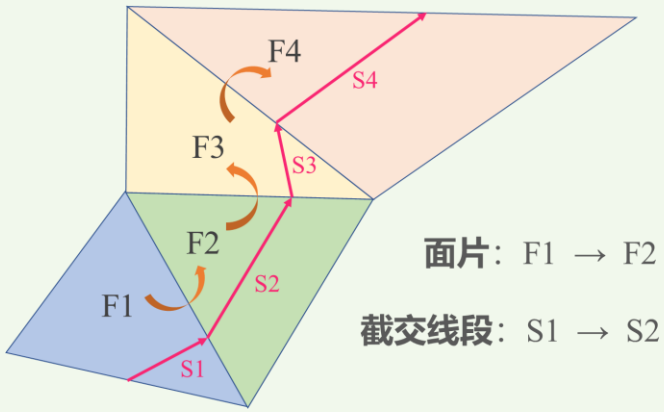
1. STL模型存在的问题

- **问题3. 模型中“面片与面片、面片与棱边、棱边与顶点” 等各几何元素之间的“相邻”关系不明确。**
- 面片与面片之间相互独立，**信息通道完全被隔绝**，无法从当前面片快速定位到与之相邻的面片（除非进行遍历搜索，但那样做将消耗大量的时间）。

口 如果可以从当前面片查询相邻面片，能做什么?

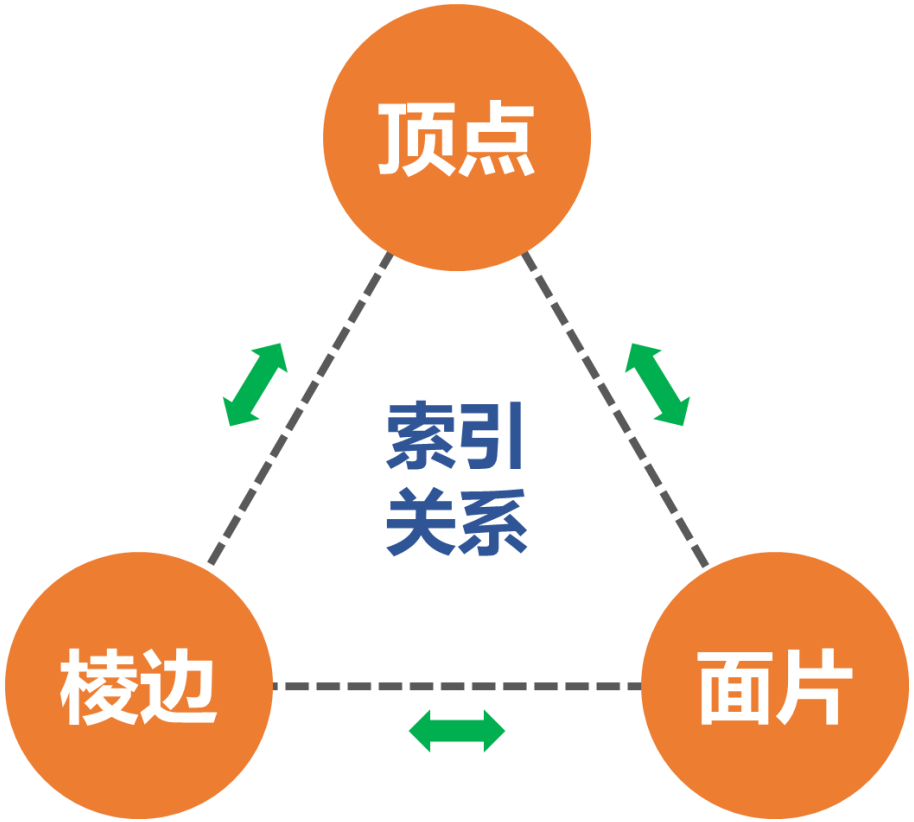
① 从任意一个面片出发，求取该面片和截平面的交线。
② 查询该面片的相邻面片，并求取相邻面片和截平面的交线。
③ 查询相邻面片的相邻面片，然后求交。
④ 依此不断迭代，直至所有面片被用完。

- 在这一过程中，从**相邻面片**求得的**截交线段**也必然是**相邻、相接**的，因此在截交过程中就可以直接对交线依次进行拼接，整个截交和拼接过程将**融为一体**。
- 如果把第一个面片称为“**种子**”，求取切片轮廓的过程类似于种子不断“**追踪**”相邻面片的过程。



1. STL模型存在的问题

- **拓扑关系定义：**
- 空间多面体中面片、棱边、顶点这些几何实体之间的 **相邻、关联、包含、索引** 等关系。



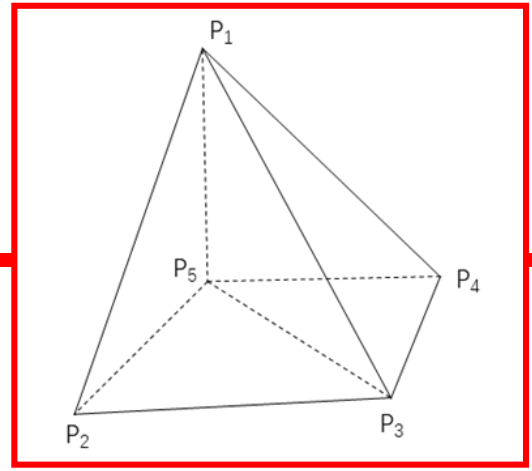
任务

根据输入STL模型，构建其

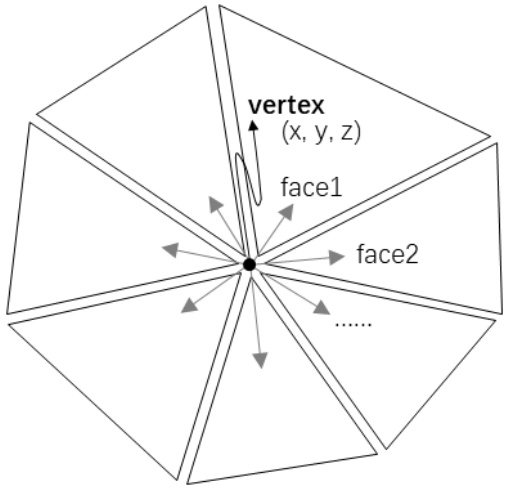
“顶点/棱边/面片”

的相互索引（拓扑）关系。

2. 拓扑关系表示（数据结构）



- **顶点拓扑结构**
  - **假设：STL模型中顶点不存在冗余情况**（为什么？）
    - 在STL模型中，**每个三角面片都有独立的顶点**，这导致坐标相同的顶点，各自拥有不同的顶点对象。这将增加建立顶点拓扑关系的难度。
    - 理想情况下，每个顶点位置上应该只有一个顶点对象，以下将在这一理想情况下继续讨论顶点拓扑结构。
  - **顶点拓扑属性**
    - **坐标：**顶点应该拥有X、Y、Z三个方向的坐标，以指示它所在的空间位置。
    - **面片：**一个顶点至少被 3 个三角面片共用，因此在顶点内部，应该有一个面片指针列表，用来保存所有用到该顶点的三角面片指针（引用）。



Vertex

坐标：**x, y, z**

面片：**faces**

棱边：**edges** X

是否需要保存棱边?

- 在STL模型中，一个顶点**被若干条棱边共用**。因此顶点理应还有一个棱边指针列表，用来保存所有用到该顶点的棱边指针（引用）。
- **规定：**顶点不保存棱边，面片保存棱边，顶点间接索引棱边。

2. 拓扑关系表示

- **顶点拓扑结构**
- 用Python语言建立一个拓扑顶点类**TVertex**，前缀T表示**Topology**，即拓扑。

TopoSlicer.py	TVertex类定义，它表示具有拓扑关系的顶点	7-1

```
1   ……                                          # 导入必要库
2   class TVertex:                              # 定义Tvertex类
3       def __init__(self, pnt3d, digits = 7):  # 从Point3D对象初始化
4           self.x = round(pnt3d.x, digits)     # 顶点的x、y、z坐标
5           self.y = round(pnt3d.y, digits)
6           self.z = round(pnt3d.z, digits)
7           self.faces = []                     # 和顶点相接的面片引用
8       def toTuple(self):                      # 将顶点转化为元组
9           return (self.x, self.y, self.z)
10      def toPoint3D(self):                    # 将顶点转化为Point3D对象
11          return Point3D(self.x, self.y, self.z)
12      def isSmaller(self, other):             # 比较两个顶点大小
13          if self.x < other.x: return True
14          elif self.x == other.x and self.y < other.y: return True
15          elif self.x == other.x and self.y == other.y and self.z < other.z: return True
16          return False
```

2. 拓扑关系表示

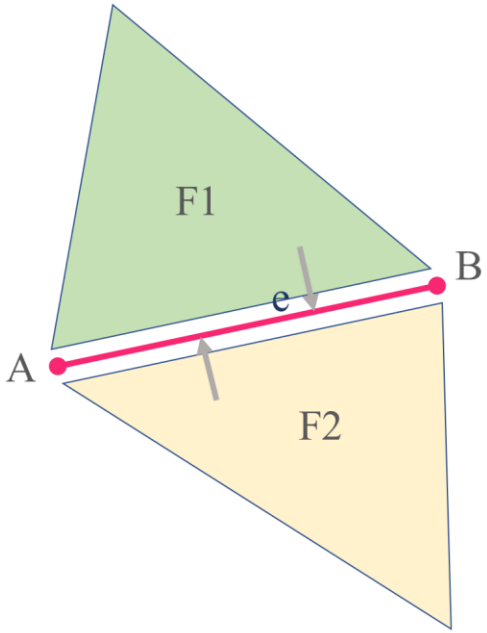
- **顶点拓扑结构**

- **__init__**初始化函数，定义了x、y、z坐标属性和一个faces列表属性，其中faces列表用于保存和当前顶点相接的所有三角面片指针。和LinkPoint类似，设置了digits参数，用于截断不必要的坐标精度，默认情况下，digits值为7。
- **toTuple**函数的作用是将一个TVertex对象的3个坐标转化为元组，这个函数在把顶点根据坐标值放到字典中用到。
- **toPoint3D**的作用是将一个TVertex对象转化为一个Point3D对象，这个函数是必不可少的，因为Point3D类封装了很多重要的几何操作函数，通过这个转化函数，TVertex对象可以间接调用这些函数。
- **isSmaller**函数，这个函数的作用是比较两个TVertex对象的大小，比较原则仍然是之前介绍过的点的字典序比较法。

为什么不在 isSmaller 函数中直接调用 cmp_pntSmaller函数?

2. 拓扑关系表示

- **棱边拓扑结构**
- 棱边**整边**结构：
  - **顶点**：一条棱边有两个不同的顶点，因此，在棱边中，应该保存它的两个**顶点的引用**。
  - **面片**：一条棱边的两侧各有一个三角面片，因此，在棱边中，也应保存和它相邻、以它为公共边的**两个三角面片的引用**。
- 整边结构**缺点**：**模棱两可**，一条棱边到底应该属于哪个面片？

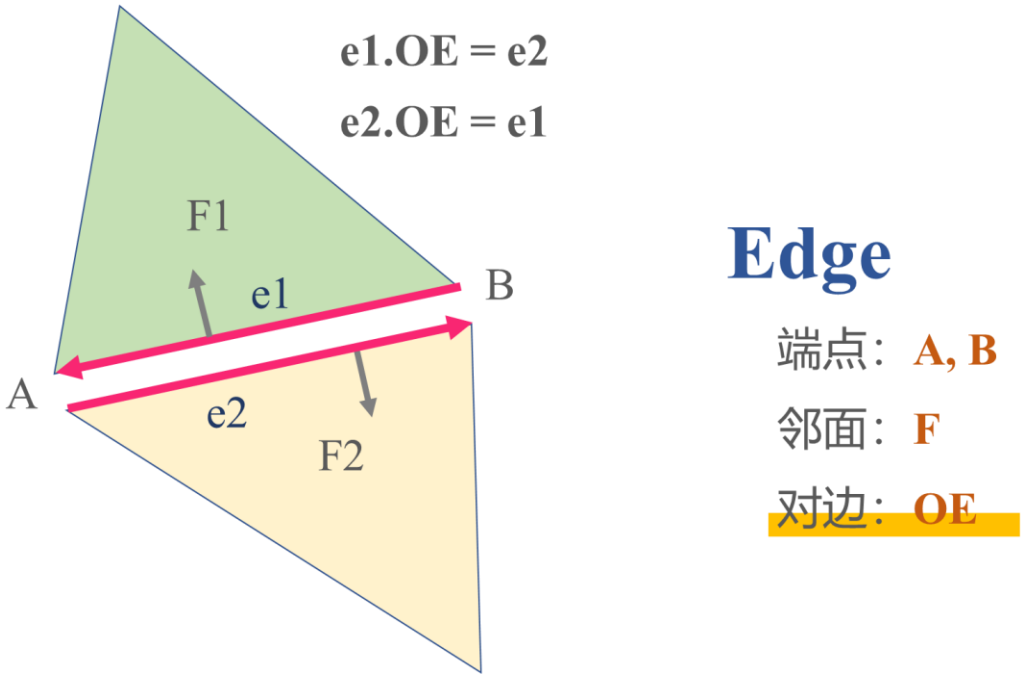


棱边 e 属于
F1 or F2 ?

2. 拓扑关系表示

• 棱边拓扑结构

- 棱边**半边**结构：将公共棱边 e 拆分成两条半边 e_1 和 e_2，它们的**位置**和**坐标**完全一致，但从属于不同的面片，其中半边 e_1 属于面片 F_1，半边 e_2 属于面片 F_2。
  - **顶点/面片**：一条半边应该都有两个**顶点的引用**，而且它还应该有一个**面片引用**，用于保存它所从属的那个三角面片。
  - **对边**：大都情况下，**半边总是成对出现**，在半边中，还有一个**半边引用**，用于保存和它分享同一棱边的另一半边，即对边。



2. 拓扑关系表示

- **棱边拓扑结构**
- 建立半边类**TEdge**，用于保存半边和顶点以及面之间的几何拓扑关系。

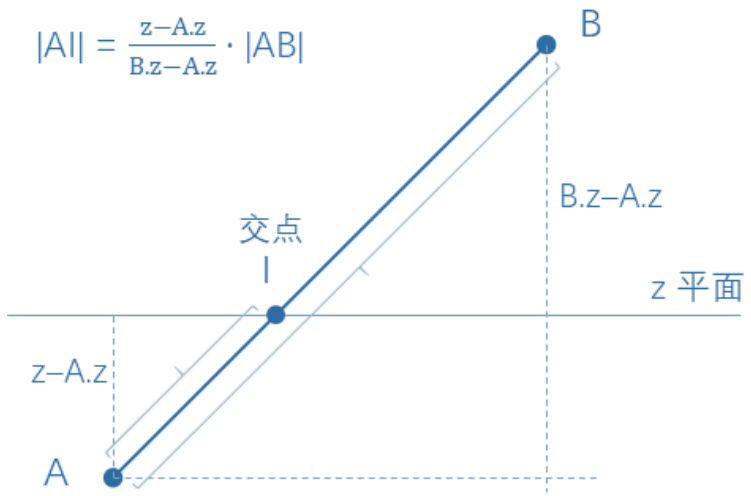
TopoSlicer.py	TEdge类定义，它表示具有拓扑关系的棱边（半边）	7-2

```
1   ……                                          # 导入必要库，省略其他代码
2   class TEdge:                                # 定义Tedge类
3       def __init__(self, tA, tB):             # 从两个TVertex点初始化
4           self.A, self.B = tA, tB             # 两个顶点引用 A、B
5           self.F = None                       # 所从属的面片引用 F
6           self.OE = None                      # 对边引用 OE
7       def toTuple(self):                      # 将边转化为元组（6维）
8           if self.A.isSmaller(self.B):
9               return (self.A.x, self.A.y, self.A.z, self.B.x, self.B.y, self.B.z)
10          else:
11              return (self.B.x, self.B.y, self.B.z, self.A.x, self.A.y, self.A.z)
12      def intersect(self, z):                 # 边和z平面求交，返回交点
13          if min(self.A.z, self.B.z) > z or max(self.A.z, self.B.z) < z:
14              return None                     # 如果边高于或低于z，无交点
15          elif self.A.z == self.B.z == z:
16              return None                     # 如果边位于z平面，无交点
17          else:
18              if z == self.A.z: return self.A.toPoint3D()
19              else:
20                  ratio = (z - self.A.z) / (self.B.z - self.A.z)      # 根据z比例计算
21                  vec = self.A.toPoint3D().pointTo(self.B.toPoint3D()).amplified(ratio)
22                  pnt = self.A.toPoint3D() + vec
23                  return pnt
```

2. 拓扑关系表示

• 棱边拓扑结构

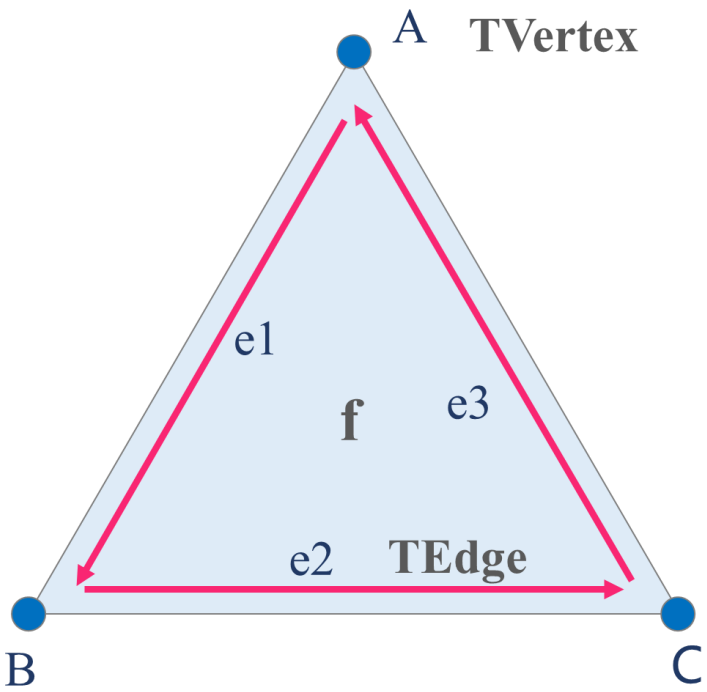
- **TEdge** 包含4个属性，分别是 **A**、**B**、**F** 和 **OE**。其中 A 和 B 为 **Tvertex** 类型的引用，表示半边的两个顶点，F 为半边所在的面片的引用，OE 为当前半边的对边的引用。这里特别强调“引用”或“指针”的概念，是为了后期应用时能快速索引这些对象。
- **toTuple** 函数作用是将两个顶点的 6 个坐标转化成一个 6 维的元组。为了保证两条位置和坐标完全一致的半边最终能够得到一样的6维元组，这里用了一个小技巧，即对TEdge的两个TVertex顶点进行比较，保证较小的点的 3 个坐标落在 6 维元组的前3位。
- **intersect** 函数的作用是计算当前半边和高度为z的XY平面（即z平面）的交点，它是后面计算拓扑模型和截平面交线的核心基础函数。（为什么不用intersectSegmentPlane函数？）



对于截平面是z平面这一特殊情况，通过非常简单的预处理就可以排除一些显然不相交的情形，这样可以显著提交相交计算效率，如半边的最低点高于截平面，或最高点低于截平面，或半边落在截平面上。

2. 拓扑关系表示

- **面片拓扑结构**
- 在建立了顶点和半边的拓扑结构之后，建立面片的拓扑结构变得相对简单。
  - **顶点**：一个面片必然包含3个顶点，须保存**3个顶点的引用**（但每个顶点并非当前面片独有，它们也有可能被其他面片共享）。
  - **半边**：一个面片必然也包含3条半边，且每条半边都是当前面片独有的，因此，在一个面片中，也必须保存**3条半边的引用**。



Face

顶点：**A, B, C**

棱边：**e1, e2, e3**

2. 拓扑关系表示

- **面片拓扑结构**
  - 建立拓扑面片类TFace，用于保存面片和顶点以及半边的几何拓扑关系。

TopoSlicer.py	TFace类定义，它表示具有拓扑关系面片	7-4

```
1   ……                                          # 导入必要库，省略其他代码
2   class TFace():                              # 定义TFace类
3       def __init__(self, tA, tB, tC, te1, te2, te3):   # 初始化函数
4           self.A, self.B, self.C = tA, tB, tC          # 顶点引用A、B、C
5           self.E1, self.E2, self.E3 = te1, te2, te3    # 半边引用 E1、E2、E3
6           self.used = False                            # 面片是否被使用过
7       def zMin(self): return min(self.A.z, self.B.z, self.C.z)  # 获取面片Z方向最低点
8       def zMax(self): return max(self.A.z, self.B.z, self.C.z) # 获取面片Z方向最高点
9       def intersect(self, z):                      # 面片和截平面求交
10          if self.zMin() > z or self.zMax() < z:   # 如果面片高于或低于截平面
11              return None, None, None              # 则返回 None, None, None
12          elif self.A.z == self.B.z == self.C.z == z:   # 如果面片和截平面重合
13              return None, None, None              # 则返回 None, None, None
14          else:
15              c1 = self.E1.intersect(z)            # 半边和截平面求交
16              c2 = self.E2.intersect(z)
17              c3 = self.E3.intersect(z)
    ……
```

2. 拓扑关系表示

- **面片拓扑结构**
- **TFace** 类包含 7 个属性，它们分别是**A、B、C、E1、E2、E3**以及**used**。
  - **A、B、C：**面片的顶点引用，它们并非单独属于某个面片，而是同时由某几个面片共享；
  - **E1、E2、E3：**分别为面片的半边引用，在半边初始化时，规定E1为由顶点A、B构成的半边，E2为由顶点B、C构成的半边，E3为由顶点C、A构成的半边，这一规定非常重要，必须遵照执行；
  - **used：**表示当前面片是否已经被使用过，该属性将在后面截交计算时用到。
- **zMin** 和 **zMax** 函数分别用于获取面片3个顶点在Z方向上极小值和极大值。
- **intersect**函数，这个函数的功能是计算当前三角面片和截平面的交线，它是后面切片轮廓计算的核心基础函数。它的实现原理和之前在GeomAlgo模块中实现的 **intersectTrianglePlane** 函数完全一样。（**为什么又重新实现一遍？**）
  - intersectTrianglePlane返回是Segment类型的截交线段或None，其中None表示三角面片和截平面不相交。
  - intersect的返回相对复杂，包括三个部分：第一部分为Segment类型的线段（或None），第二部分和第三部分为交线两个端点所在的三角面片的半边或顶点引用。如果交线的两个端点都在三角面片的半边上，则在第二部分返回两条半边的引用；如果交线的两个端点分别在三角面片的一条半边和一个顶点上，则在第二部分返回半边的引用，第三部分返回顶点的引用。如果面片和截平面不相交或只交于一点，则返回的三部分均为None。

3. STL拓扑模型重建

- **拓扑重建动机**
  - STL模型中顶点冗余，每个顶点平均被保存了6次;
  - STL模型中顶点、棱边、面片之间的拓扑关系不明确;
  - 方便对STL模型切片，获得切片轮廓。
- **拓扑重建目标**
  - 从当前任一顶点、半边或面片出发，能够快速索引与之相邻的其他顶点、半边或面片，继而可以迭代索引到整个模型中任意的顶点、半边或面片。（由一及百、链式反应）
- **拓扑重建思路**
  - 利用 **TVertex**、**TEdge**、**TFace** 这3个类;
  - 消除冗余顶点，构建顶点、棱边、面片索引关系。



3.STL拓扑模型重建

- **冗余顶点消除**

- **问题背景**：对输入的STL模型，如果从每个独立的三角面片中收集顶点坐标，并把这些顶点全部保存到一个**顶点列表**（数组）中，根据欧拉定理，顶点列表中大概有**六分之五**的元素的冗余的，具有相同坐标的顶点被不断重复保存，只有六分之一的顶点是有用的。

- **问题抽象**：对一个可能保存了相同元素的列表，如何以最快的速度得到它的**unique列表**。这里unique列表的意思是列表中的元素各不相同。

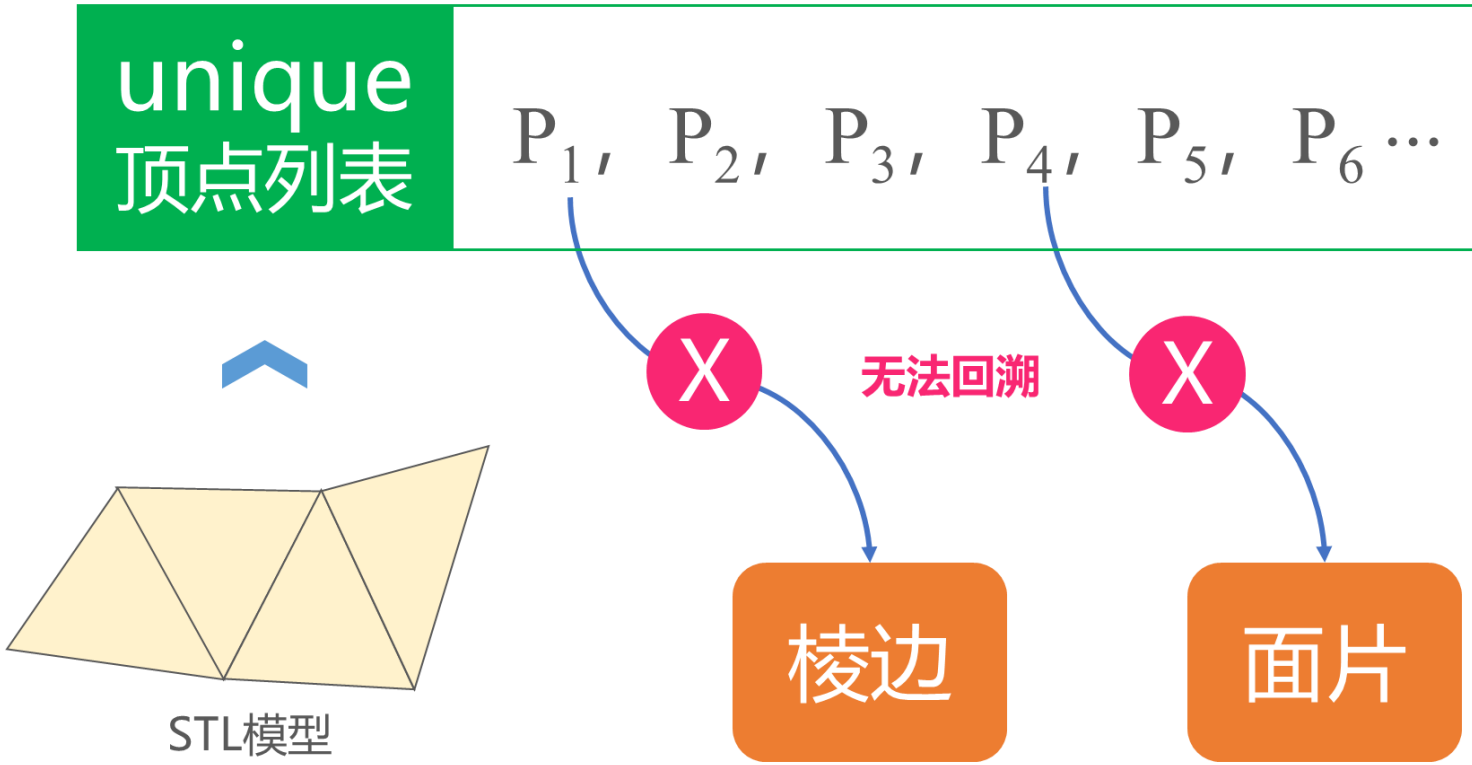


一种思路：排序以消除相同元素

- 排序完成后，在列表中，具有相同值的元素被排在的相邻的位置上。
- 从列表的第一个元素开始，逐个比较相邻元素对的值是否相等，如果相等，则移除其中一个元素，直至列表末尾。
- 时间复杂度O(nlog(n)).

3.STL拓扑模型重建

- **冗余顶点消除**
- 排序法确实能够消除冗余顶点，得到一个毫无冗余的顶点列表之后。但**无益于顶点、棱边、面片的拓扑关系建立**。
  - 排序过程会完全打乱顶点顺序，很难回溯哪个顶点属于哪个面片、哪条棱边。



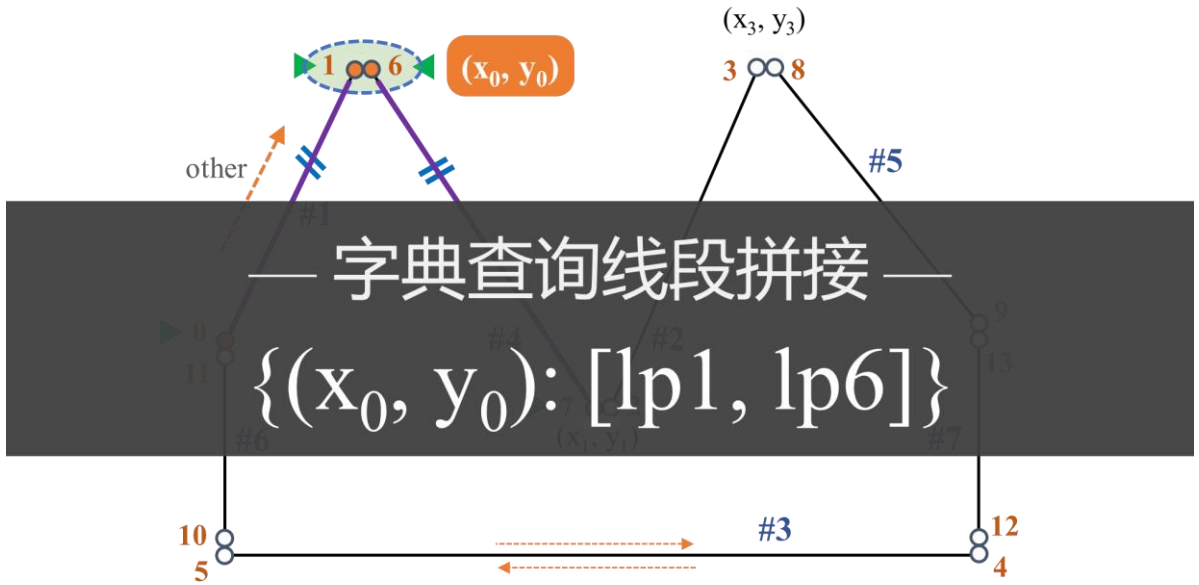
3.STL拓扑模型重建

• 冗余顶点消除

- 在拓扑重建背景下，消除冗余顶点的**最佳时机**应是在还能理清一个三角面片内面、边、顶点三者关系的时候，这样至少能很快建立面内半边与顶点、面片与顶点、面片与半边之间的拓扑关系。

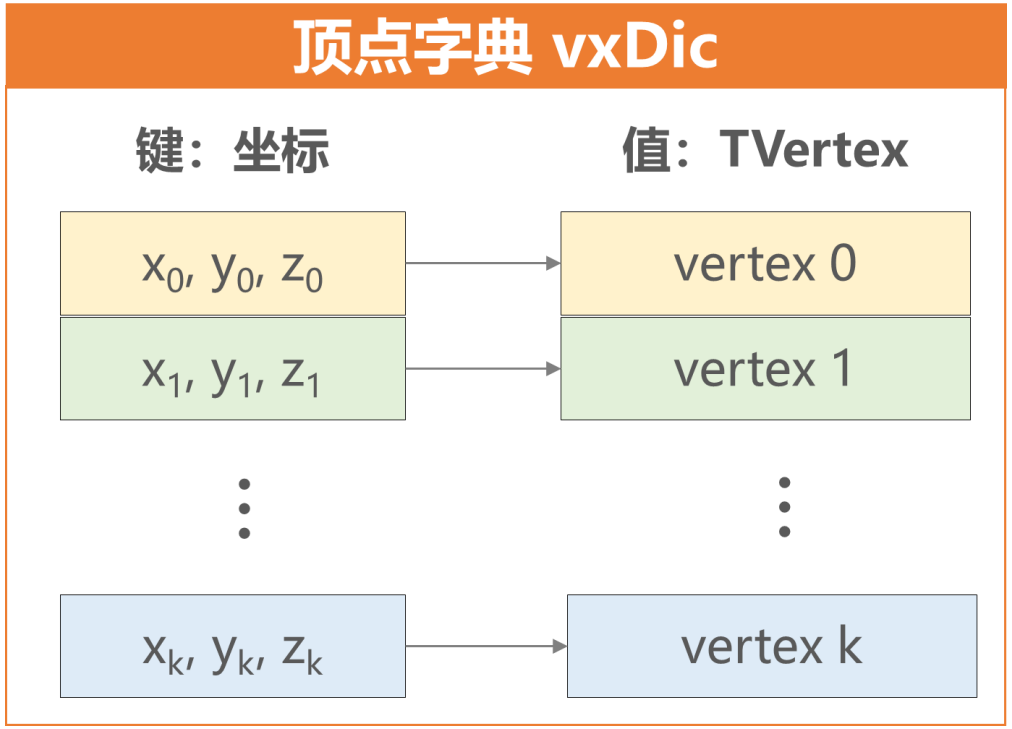
类似案例：

- 在基于字典查询的拼接优化方法中，将具有相同坐标位置的链接点（LinkPoint）放在了**字典**结构的同一个键下面，从而实现相邻截交线段的快速拼接。这个过程实现的功能从另一个层面来说其实就是寻找并收集冗余顶点。



3.STL拓扑模型重建

- **冗余顶点消除：基于字典**
- 使用字典存放STL模型中的顶点，记为**vxDic**，字典中：
  - **键**：顶点坐标，即通过坐标可从字典中找到相应的顶点对象（引用）；
  - **值**：顶点本身，或更贴切地说，是顶点引用。



3.STL拓扑模型重建

- **冗余顶点消除：基于字典**

- 对STL模型中的**任意一个三角面片ABC**，记它的顶点（TVertex类型）分别为A、B、C，对应坐标分别为(x_A, y_A, z_A)、(x_B, y_B, z_B)、(x_C, y_C, z_C)。

  - 对顶点A，首先在字典vxDic中查询以(x_A, y_A, z_A)为键的坐标是否已经存在：
    - **如果不存在，则将(x_A, y_A, z_A)和A以键值对添加至vxDic；**
    - **如果vxDic中已经存在(x_A, y_A, z_A)键，则说明之前有坐标同为(x_A, y_A, z_A)的顶点已经被添加进了字典，不去执行任何操作，让具有相同坐标的点只在字典中保存一次。**
  - 对顶点B和C，按以上步骤执行相同的操作。

- 借助字典vxDic，对STL模型中的每个三角面片不断重复上述过程，就可以消除冗余顶点。

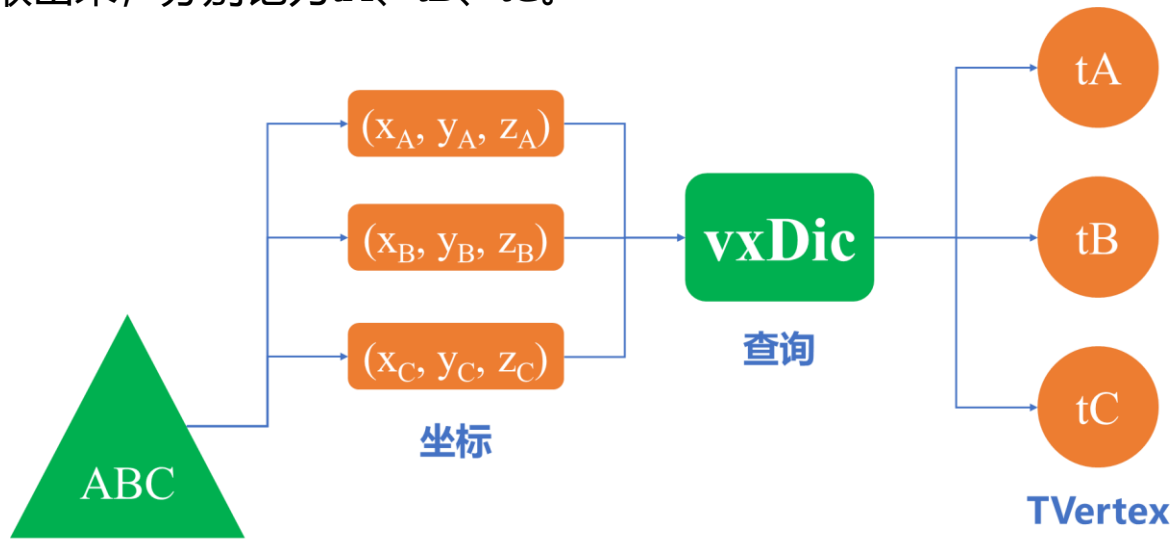
- 由于字典查询的时间复杂度为O(log(n))，因此使用字典消除冗余顶点总的时间复杂度为O(nlog(n))，其中n为STL模型总顶点数，包含冗余顶点。

- **优势**：**以三角面片为单元消除冗余顶点，同时可整理顶点、棱边、面片拓扑关系**。

3.STL拓扑模型重建

- **拓扑关系建立：顶点获取**

- 对三角面片ABC，在执行完字典添加操作后，可从**字典vxDic**中把位于(x_A, y_A, z_A)、(x_B, y_B, z_B)、(x_C, y_C, z_C)三点处的值（顶点引用）重新取出来，分别记为tA、tB、tC。

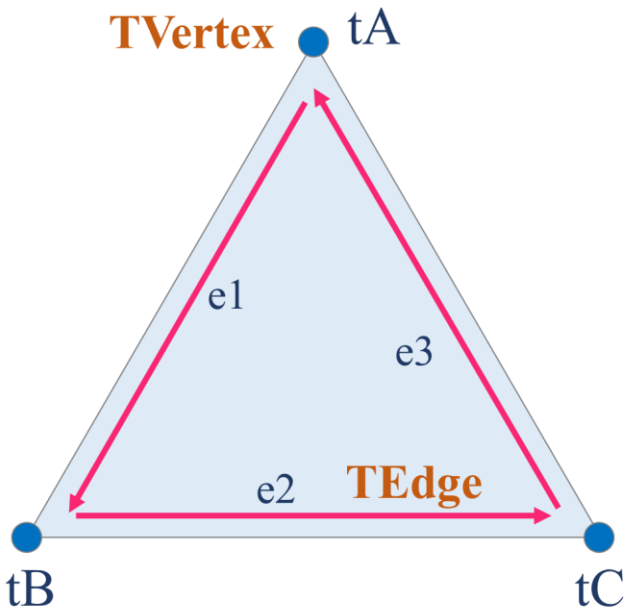


tA、tB、tC 和 A、B、C 有什么关系?

- 以顶点tA为例，在计算机内存中，tA和A可能指向同一块内存地址，也可能指向不同的内存地址，如果它们的地址相同，说明顶点tA刚被添加进了字典vxDic，如果它们的地址不同，则说明tA在前面的操作中已被添加进了vxDic。

3.STL拓扑模型重建

- **拓扑关系建立：**
- **棱边构建**：根据以上顶点 tA、tB 和 tC，建立当前三角面片的3条半边（**TEdge**类型），分别记为 e_1、e_2、e_3，其中：
  - e_1的两顶点为tA和tB
  - e_2的两顶点为tB和tC
  - e_3的两顶点为tC和tA



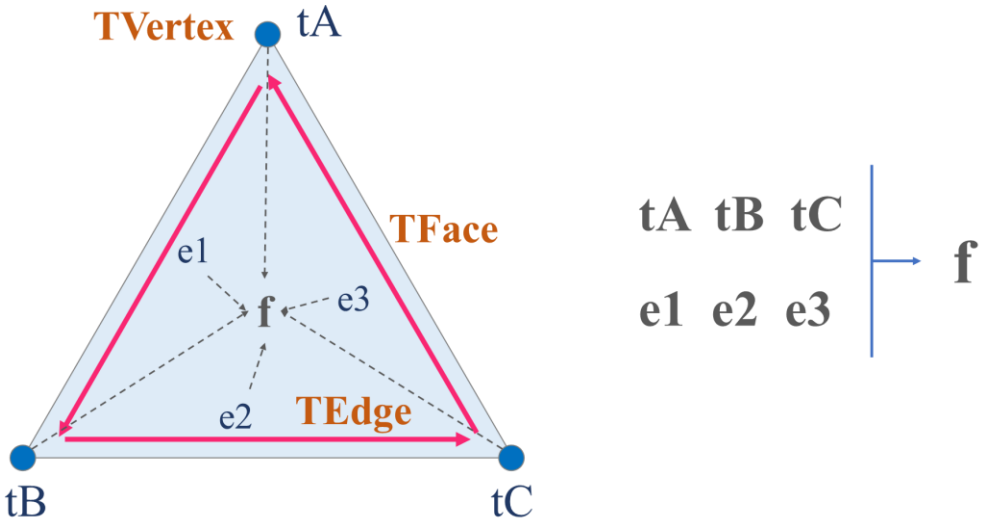
e1: tA → tB

e2: tB → tC

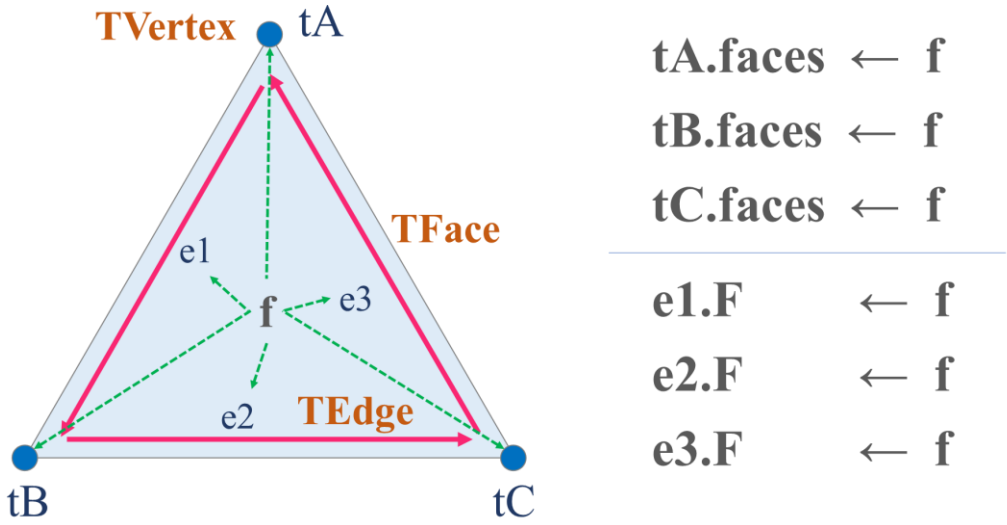
e3: tC → tA

3.STL拓扑模型重建

- **拓扑关系建立：**
- **面片构建**：根据顶点 tA、tB、tC 和半边 e_1、e_2、e_3，建立当前面片的 **TFace** 对象，记为 f。
- 基于 f：
  - 一方面，对顶点 tA、tB、tC 的 **faces** 属性赋值，将 f 分别添加至各顶点的 faces 列表，表示面片 f 被这些顶点共享。
  - 另一方面，对半边 e_1、e_2、e_3 的 **F** 属性赋值为 f，表明这些半边是当前面片f的半边。



构建面片f

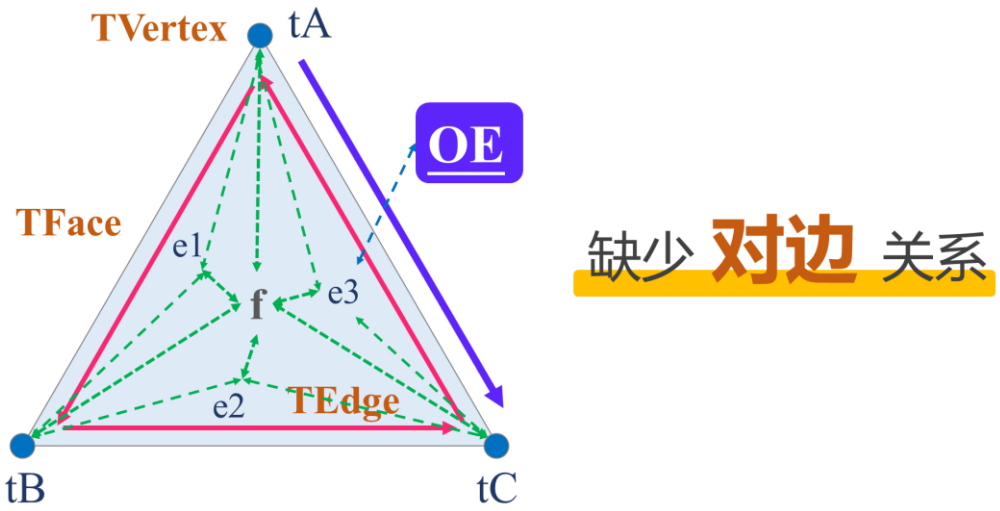


对顶点和半边的赋值

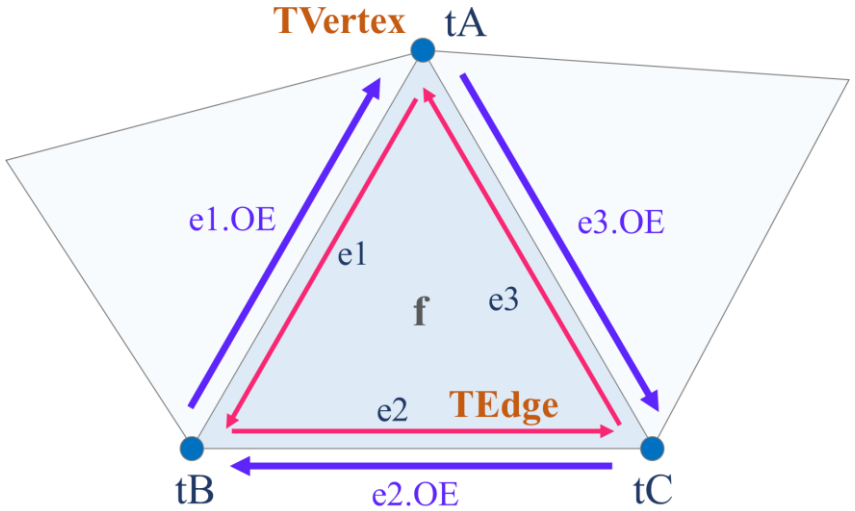
3.STL拓扑模型重建

拓扑重建难点

以上拓扑关系都是在单个三角面片内建立的，**如何建立两个面片之间的棱边成对对边关系?**



缺少 **对边** 关系

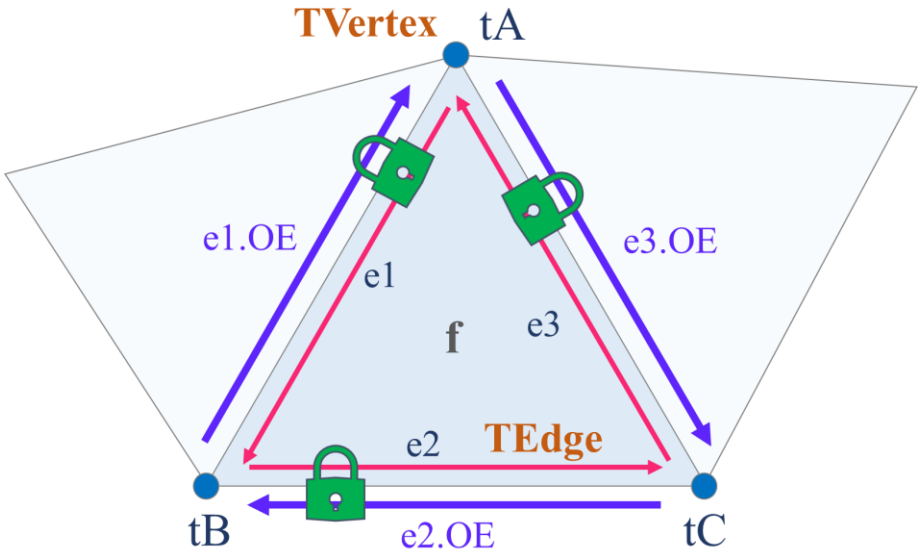


为什么要建立对边拓扑关系?

如果不建立，从当前面片查询与之相邻的面片会比较费劲，不得不通过顶点的拓扑关系来筛选，即从顶点的faces属性中遍历寻找相邻面片。

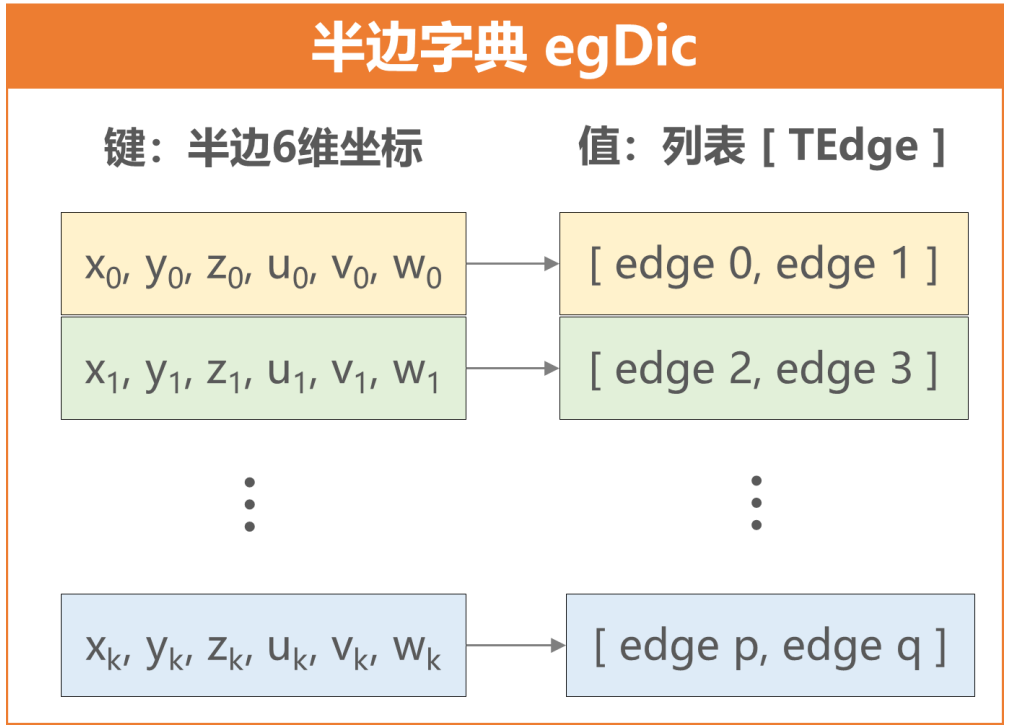
3.STL拓扑模型重建

- **拓扑关系建立：对边关系建立（难点）**
- 大多时候，半边总是成对出现的，它们虽然分享了两个相邻三角面片的公用顶点，却各自属于不同的面片。要想构建成对半边之间的拓扑关系，首先得**找出所有的成对半边**。
- **对边特征**：来自同一条整边，重合，即**两端点坐标相同**，但顶点顺序不确定（无序）。
- **对边建立思路**：借鉴冗余顶点消除方法，使用字典，将半边添加进字典前，查询是否已有相同的“**键**”在字典中：
  - 如果没有，则添加半边
  - 如果有，则组队



3.STL拓扑模型重建

- **拓扑关系建立：对边关系建立（难点）**
  - **字典的键**：每个半边有两个顶点，无规律存放，先对这两个顶点排序（字典序，如小的顶点在前，大的顶点在后），得到确定的6维标识，确保两对边坐标一致。TEdge中的**toTuple**函数已经实现了这个功能，将半边转化为唯一的**6维标识**。。
  - **字典的值**：半边列表。多数情况下，列表中的半边数应为2，即成对半边；如果列表中半边数为1，表明STL模型在此处不封闭。



3.STL拓扑模型重建

- **拓扑关系建立：对边关系建立**

- 建立半边字典**egDic**，字典的键为半边的 6 维坐标，**值**为半边列表。
- 对STL模型中任意三角面片ABC，在得到它的半边 e_1、e_2、e_3 之后，分别检查 e_1、e_2、e_3 的 6 维坐标是否在 egDic 中。以半边 e_1 为例说明：
  - 如果e_1的6维坐标不在egDic中，则以e_1的6维坐标和包含e_1一个元素的半边列表为键值对加入字典egDic中；
  - 反之，如果e_1的6维坐标已经在egDic中，则直接将e_1添加至它的6维坐标对应的半边列表中。

```
key = ( x1, y1, z1, u1, v1, w1 )
val = e1
if key not in egDic.keys:
    egDic[key] = []
egDic[key].append(val)
```

- 当遍历完STL模型中的所有三角面片，对得到的半边字典egDic，遍历字典中的每个值，即每个列表：
  - 如果列表中有两个元素，将列表中的0号半边的对边（即TEdge的OE属性）设置为1号半边，并将1号半边的对边设置为0号半边。
  - 如果列表中只有一个元素，即一条半边，不做任何处理，这表明模型在此处未封闭。

```
for edges in egDic.values:
    if len(edges) == 2:
        edges[0].OE = edges[1]
        edges[1].OE = edges[0]
```

3.STL拓扑模型重建

- **拓扑关系建立：代码实现**
- 新建TModel的类，用于表示拓扑模型，并在类中封装拓扑模型重建函数。

TopoSlicer.py	TModel类定义，它表示拓扑模型，封装拓扑模型重建函数	7-5

```
1   ……                                                          # 导入必要库
2   class TModel:                                               # 定义TModel，拓扑模型
3       def __init__(self, stlModel):                           # 根据输入STL模型初始化
4           self.vxDic, self.egDic = {}, {}                     # 顶点字典，半边字典#
5           self.faces = []                                     # 面片列表
6           self.stlModel = stlModel                            # 保存STL模型
7           self.createTModel()                                 # 调用模型拓扑重建函数
8       def createTModel(self):                                 # 核心函数：模型拓扑重建
9           for t in self.stlModel.triangles:                   # 遍历STL模型中面片t
10              A, B, C = TVertex(t.A), TVertex(t.B), TVertex(t.C) # 根据Point3D创建TVertex
11              if A.toTuple() not in self.vxDic.keys():        # 检查顶点A是否在vxDic中
12                  self.vxDic[A.toTuple()] = A                 # 如果不在，则添加
13              if B.toTuple() not in self.vxDic.keys():        # 检查顶点B
14                  self.vxDic[B.toTuple()] = B
15              if C.toTuple() not in self.vxDic.keys():        # 检查顶点C
16                  self.vxDic[C.toTuple()] = C
17              tA = self.vxDic[A.toTuple()]                    # 从vxDic中根据A坐标查询
18              tB = self.vxDic[B.toTuple()]                    # 保存的顶点引用：tA、tB、tC
19              tC = self.vxDic[C.toTuple()]
20              e1, e2, e3 = TEdge(tA, tB), TEdge(tB, tC), TEdge(tC, tA) # 创建半边e1、e2、e3
21              f = TFace(tA, tB, tC, e1, e2, e3)               # 创建面 f
    ……
```

4.拓扑模型切片

- **拓扑关系建立：代码实现**
  - TModel类的属性：
    - **vxDic** 为顶点字典
    - **egDic** 为半边字典
    - **faces** 为面片列表，用于保存根据STL模型构建的拓扑面片
    - **stlModel** 用于保存输入的需要重建的STL模型
  - 根据建立的拓扑关系，理论上，从任意一个面片出发，顺藤摸瓜，可以遍历模型中所有三角面片。
    - 对于任意一个面片 f，可快速索引到它的 3 个顶点和 3 个半边。如：f.A、f.B、f.C 以及 f.E1、f.E2、f.E3，可以通过类似 f.E1.OE.F 的形式快速索引到与之相邻的面片。
    - 对任意半边 e，可以快速索引到它的两个端点、所在的面以及对边。如：e.A、e.B、e.F 以及 e.OE。
    - 对任意顶点 A，可以根据 faces 列表，索引到以之为顶点的所有面片。

- **问题：如何从当前棱边寻找同一个面片内的下一条棱边?**

4.拓扑模型切片

如果可以从当前面片查询相邻面片，能做什么？

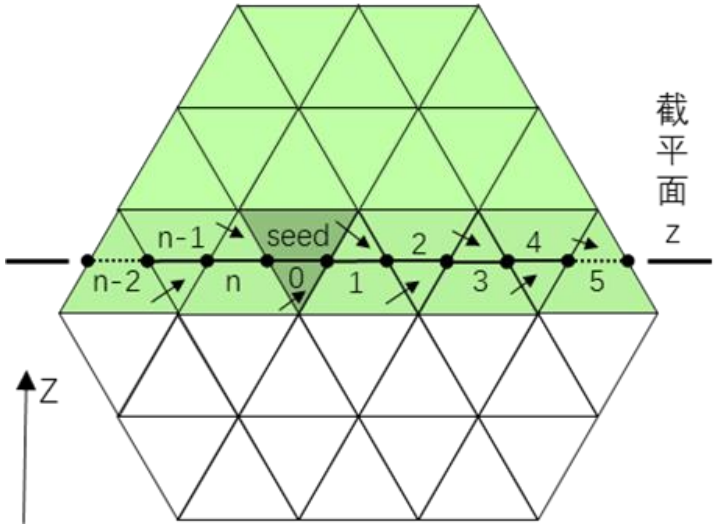
（1）从任意一个面片出发，求取该面片和截平面的交线。

（2）查询该面片的相邻面片，并求取相邻面片和截平面的交线。

（3）查询相邻面片的相邻面片，然后求交。

（4）依此不断迭代，直至所有面片被用完。

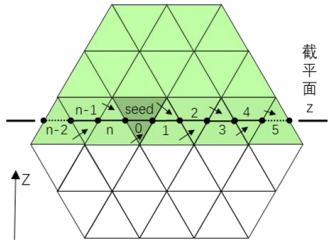
- **基本现象**
  - 在切片过程中，如果当前截平面和STL模型中的其中一个三角面片相交，那么截平面也必然会和这个面片的其中一个（或两个）相邻面片相交。
- **基本思想**
  - 首先，找到和截平面相交的**第1个面片**。
  - 根据顶点、面片、棱边之间的拓扑关系，顺藤摸瓜，不停**追踪**临近面片，从而找出拓扑模型中和截平面相交的所有面片。
  - 在追踪过程中，由相邻、相接面片求得的截交线段也必然是相邻、相接的，即这些截交线段是天然有序排序的，而并非是散乱无序的，这样可以**省去线段拼接环节**。



首个面片很重要：**种子**

如何寻找种子面片?

4.拓扑模型切片



- **寻找种子面片（Finding Seed）**
- **问题描述**：设当前的截平面高度为z，在拓扑模型中寻找一个面片，即种子面片，使其满足和截平面相交。
- **暴力寻找**：遍历拓扑模型中的所有面片，检查当前面片是否和截平面相交，如果相交，则设该面片为种子面片，并退出遍历循环。

  ——判交而非求交

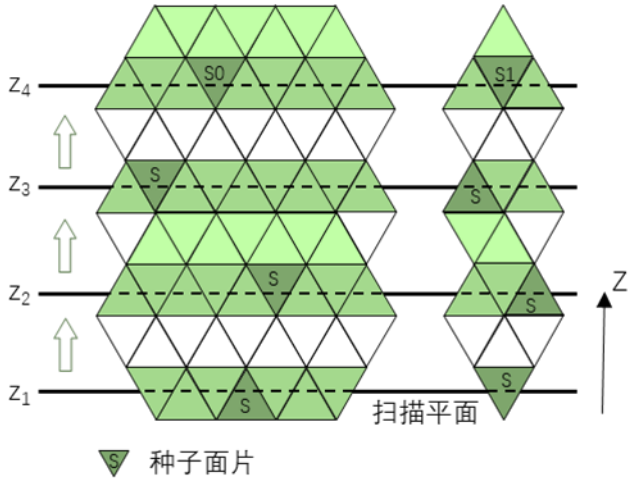
  - 在以上遍历过程中，不用去求每个面片和截平面的截交线，因为这样可能消耗大量的计算时间。
  - 只需判断每个片面的最低点和最高点组成的范围[zMin, zMax]是否包含截平面高度z值，如果包含，说明当前面片可能和截平面相交
- **暴力法的问题**
  - 寻找一个种子需要遍历模型中所有面片，效率低!
  - 如果当前截平面上有多条截交轮廓，那么需要遍历多次模型以找出种子。
- **注意**：在当前高度上，一个三角面片和截平面至多相交一次，因此，计算中，如果三角面片和截平面相交，则设置面片的used属性为True，表示其已使用过。为什么?

实例测试

- 对于包含约33万个三角面片的STL模型（monk模型），在切片后，平均每层得到的截交线段条数约为988条，即在每个切片层上，每张截平面平均约和998个三角面片相交。
- 如果按照以上方法直接在拓扑模型中寻找种子面片，那么每次为了这998个**有效**的面片，不得不去遍历33万个无关面片，即每次都多做了**330倍的无用功**。
- 由此可见，直接从拓扑模型中寻找种子面片并不是一种最优的方式。

4.拓扑模型切片

- **寻找种子面片（Finding Seed）**
- **分而治之**：在某个指定切片高度上，截平面仅和有限个三角面片相交，因此，可事先把与当前截平面可能相交的三角面片全部找出来，然后在这些面片中寻找种子面片。
  - 针对monk模型，可以把搜索范围从33万个面片减小到1000左右，这是非常大的进步！
- **扫描平面法（基本思路）**
  - 首先对模型中的面片按照最低点从低到高排序，然后构造一张扫描平面按切片层高从低到高扫描；
  - 每到一处层高，首先从扫描平面中移除已经不再与之相交的三角面片，然后再添加新的与之相交的三角面片；
  - 扫描平面总是维护着模型中与之相交的所有面片。采用排序和增量的方式维护面片数据，执行效率非常高。



扫描平面法分治面片加速种子面片寻找原理

仿照扫描平面法流程：

- 首先根据输入的切片厚度生成层高数据列表zs，然后让扫描平面沿着zs中的层高值从下往上扫描。
- 在每一层高处，扫描平面将收集和维护与之相交的所有三角面片，然后可以从这些三角面片中寻找第一个种子面片、第二个种子面片、第三个种子面片……直至这些面片的used属性均被设置为True，从而找出当前层高处的所有截交轮廓线。

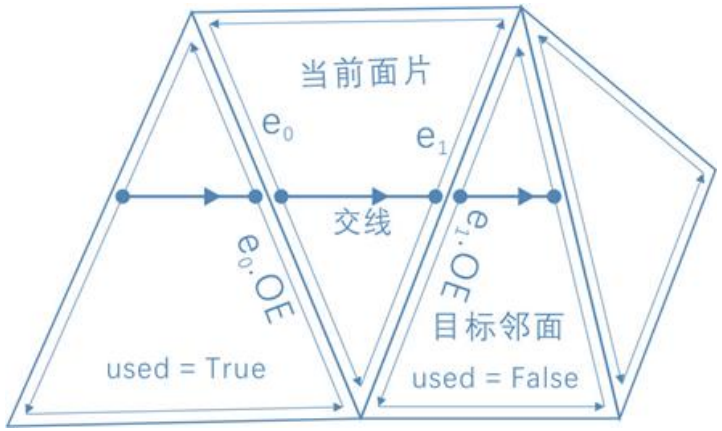
通过这种方式，可以对拓扑模型中的面片实现“分治”，从而提高种子面片的搜索效率。

4.拓扑模型切片

- **寻找目标邻面（Finding Next）：追踪**
- 在扫描平面法这一大框架下，在某一层高上，记和当前截平面相交的所有三角面片为**faces**。
  - 注意：faces为一存放TFace类型数据的列表，包含的面片数量远远小于整个模型的面片数量，甚至可以忽略不计。
  - 可直接对faces进行遍历，从中寻找从未被使用过（即used属性为False）的面片作为种子，并**将种子面片作为当前面片**。
- **相邻面片**：和当前面片相邻的面片，包括3个共用棱边的面片和多个共用顶点的面片。
  - “追踪”的下一个面片必然在当前面片的相邻面片中。但直接从这么多面片中寻找下一个面片范围仍然太大!
- **目标邻面**：必然和截平面相交的当前面片的下一个面片。
- **目标领面寻找思路**：定位当前截平面和当前面片相交的**半边**或**顶点**。
  - 假设它们的交点在两条半边上，那么只需根据其中一条半边索引它的对边，进而索引对边所在的面片，即可找到目标邻面。
  - 假设它们的交点一个在半边上，一个在顶点上呢?
  - 假设它们的交点在两个顶点上？（这种情况会出现吗？参见代码7-4）
- **TFace类**的**intersect**函数功能：对截平面z和拓扑面片进行求交，并且能够同时返回截交线段和交点所在的半边或顶点。
  - 记交点所在的半边为edges，顶点为node。这里edges为一存放TEdge类型数据的列表，它可能包含两元素或一个元素。
  - **情况1**：当edges包含两个元素时，node必为None，此时截平面和面片交线的两个端点均落在半边上；
  - **情况2**：而当edges包含一个元素时，node不为None，此时截交线段的两个端点分别落在半边和顶点上。

4.拓扑模型切片

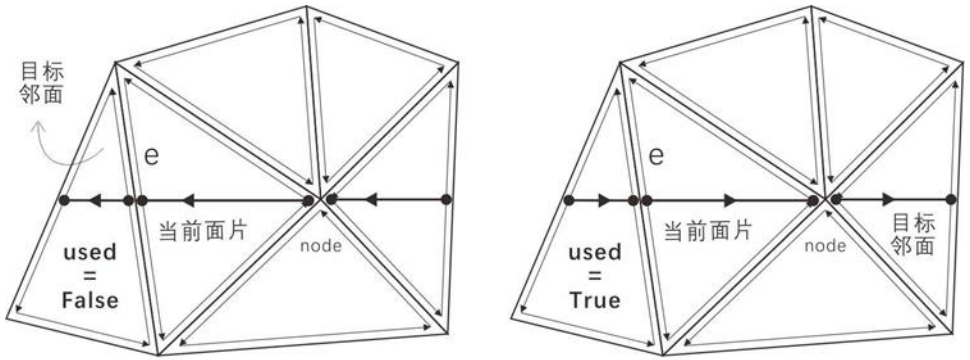
- **寻找目标邻面（Finding Next）**
- **情况1：两个交点都在棱边上**
- 从**intersect**函数返回的**edges**中取出这两条半边，并记e_0 = edges[0]，e_1 = edges[1]。显然，此时目标邻面必然在e_0对边所在的面或e_1对边所在的面当中，即e_0.OE.F或e_1.OE.F。
- 可在这两个邻面中任选一个面作为目标邻面，但前提是邻面没有使用过，因此：
  - 如果e_0.OE.F.used为False，就选e_0.OE.F作为目标邻面；
  - 如果e_0.OE.F.used为True且e_1.OE.F.used为False，就选e_1.OE.F为目标邻面；
  - 如果e_0.OE.F.used和e_1.OE.F.used均为True，尽管这在现实情况下不太可能发生，但还是要把这种情况枚举出来并给出相应对策，事实上，这只能说明当前面片的目标邻面为None，即不存在。



交线端点都在半边上时目标邻面确定原则

4.拓扑模型切片

- **寻找目标邻面（Finding Next）**
- **情况2：两个交点一个在棱边上，一个在顶点上**
- 从intersect函数返回的edges中取出这条半边，并记e = edges[0]。
- 当前面片的目标邻面极有可能就是e的对边所在的面片，即e.OE.F。
  - 如果e.OE.F.used为False，即e.OE.F从未被使用过时，那么当前面片的上一个面片必然和当前面片在顶点node处相邻，即轮廓线从上一面片穿过node而来，穿越当前面片，在半边e处穿出当前面片进入目标邻面，在这种情况下，目标邻面必然是e.OE.F。
  - 如果e.OE.F.used为True，此时目标邻面必然在以顶点node为公共顶点的面片中，此时需要遍历这些面片，找出除当前面片以外和截平面相交的面片，并把这个面片作为目标邻面，当然如果找不到这样的面片，那就说明当面面片的目标邻面为None。



(a) e.OE.F.used为False　　(b) e.OE.F.used为True

交线端点分别在半边和顶点上时目标邻面确定原则

4. 拓扑模型切片

- **寻找目标邻面（Finding Next）**
- 在TopoSlicer类将目标邻面寻找的原理和功能封装成findNextFace和findSeedFace函数，具体实现如下：

TopoSlicer.py　TopoSlicer类定义，拓扑模型切片类。部分代码，实现目标邻面寻找　7-6

```
1   ……                                          # 导入必要库，省略其他代码
2   class TopoSlicer:                           # 定义TopoSlicer类
3       def __init__(self, stlModel, layerThk):     # 根据STL模型和切片层高初始化
4           self.stlModel = stlModel            # 保存STL模型
5           self.layerThk  = layerThk           # 保存切片层高
6           self.topoModel = TModel(stlModel)   # 将STL转化为拓扑模型
7           self.layers = []                    # 用于保存生成的切片轮廓
8           self.slice()                        # 调用切片函数
9       def slice(self):                        # 定义切片核心函数
10          pass
11      def findSeedFace(self, faces):          # 定义种子面片寻找函数
12          for face in faces:
13              if face.used == False: return face
14          return None
```

……

4. 拓扑模型切片

- **寻找目标邻面（Finding Next）**
- **TopoSlicer**类中**findNextFace**函数具体实现如下：

……

```
15      def findNextFace(self, edges, node):                    # 定义目标邻面寻找函数
16          nextFace = None                                     # 记返回面片为nextFace
17          if node is None:                                    # 情形1. 交点在两半边
18              e0, e1 = edges[0], edges[1]
19              if e1.OE is not None and e1.OE.F.used == False:     # 检查e1对边的面是否已用
20                  nextFace = e1.OE.F                          # 我们是有意先检查e1的
21              elif e0.OE is not None and e0.OE.F.used == False:   # 检查e0对边的面是否已用
22                  nextFace = e0.OE.F
23          else:                                               # 情形2. 交点在半边和顶点
24              e = edges[0]
25              if e.OE is not None and e.OE.F.used == False:   # 检查e对边的面是否已用
26                  nextFace = e.OE.F                           # 如果未用，则目标邻面为该面
27              else:                                           # 否则检查和node相接的面
28                  for f in node.faces:                        # 遍历node相接面
29                      if f is not e.F:                        # 待检查面不能是当前面
30                          seg, egs, n = e.F.intersect(node.z) # 计算待检查面和z平面交线
31                          if seg is not None:                 # 存在交线，则目标邻面为该面
32                              nextFace = f
33                              break
34          return nextFace                                     # 返回目标邻面
```

4.拓扑模型切片

- **寻找目标邻面（Finding Next）**
- **TopoSlicer**类的初始化函数：对输入的STL模型，首先将它转化为TModel类型的拓扑模型，在TopoSlicer类中，接下来所有的操作均针对这个拓扑模型展开。
- **layers**属性为用于保存Layer类型数据的列表，即切片列表。
- **slice**函数为TopoSlicer类中的核心切片函数，它在类的初始化函数中被触发启动，在这里只对slice函数进行了声明，并未具体实现。
- **findSeedFace**函数功能是在扫描平面法的框架下在和当前截平面相交的三角面片（即参数faces）中寻找种子面片。
  - 它的实现原理非常简单，直接遍历输入的faces，寻找首个未被用过的面片并返回，如果所有faces均已被用过，则返回None，表示找不到种子面片。因为faces尺寸不会太大，因此不必在意findSeedFace函数的执行效率。
- **findNextFace**函数功能当然是根据当前面片寻找下一个目标邻面。但这个函数的输入并非为当前面片，而是当前面片和截平面相交后得到的交点所在的半边和顶点，即edges和node。然后对edges和node分类讨论，包括交点均在半边上以及检点分别落在半边和顶点上两种情况，从而找出目标邻面。
  - 针对第2种情况，当目标邻面和当前面片通过node相连时，调用了TEdge的intersect函数对node中存储的所有面片（node.faces，当前面片除外）进行遍历求交，那这里为什么要去执行费时的求交操作而不直接判断面片最低点zMin、最高点zMax和截平面高度z的关系呢？因为zMin ≤ z且zMax ≥ z是面片和截平面相交的必要非充分条件，当面片的一个顶点落在截平面上时，认定它们是不相交的。所以，这里调用intersect函数，确保万无一失。
  - 在intersect函数中，使用了node的z坐标作为当前截平面的高度，这是因为在这种情况下，截平面的高度确实恰好等于node的z坐标，这样做的好处是可以为findNextFace函数节省一个必要的截平面高度的输入参数。

4.拓扑模型切片

万事俱备，只欠流程

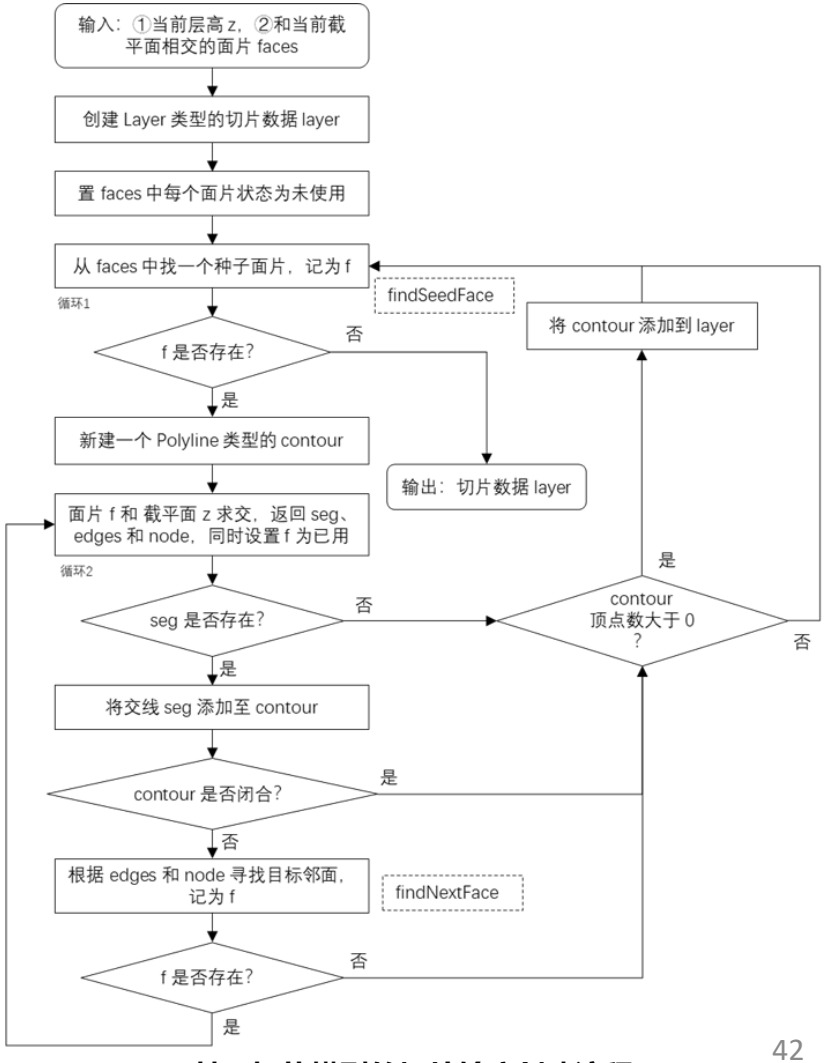
- **切片轮廓创建：指定截平面上**
- **输入**：截平面高度z，面片列表faces。
- **输出**：包含了当前层切片轮廓信息的layer。
- **对faces清洗**：将faces中每个面片的used属性值置为False。 ——**非常重要！**
- **两层循环**

- 第1层循环主要做的就是从faces中找出一个种子面片：如果找不到种子面片，表明faces中所有面片均已被使用，此时可以输出切片数据layer，并退出整个流程；如果能够找到种子面片，则新建一条Polyline类型的轮廓做为当前轮廓contour，并进入第2层循环。
- 第2层循环的核心功能就是不断根据当前面片寻找与之相邻的下一面片，并在这一过程中，把截交得到线段添加到当前轮廓。

- **退出条件（第2层循环）**

- （1）当前面片和截平面不相交，这个条件在正常情况下似乎很难满足，除非有现在还没预料到的异常发生，这是因为在第2层循环中，通过findNextFace函数寻找当前面片，如果它找到的面片和截平面不相交，那它就直接返回None。为了保证流程的稳定性和鲁棒性，在流程中保留这个退出条件。
- （2）当前轮廓contour已封闭，这是理所当然的正常退出，当前轮廓封闭后，应该再寻找一个种子面片。
- （3）根据当前面片找不到目标邻面，即findNextFace返回None，这表明模型在当前面片处未封闭或存在裂缝，这是不正常的，也是不应该发生的，如果在循环中遭遇了这个退出条件，将得到一条非封闭的轮廓线。

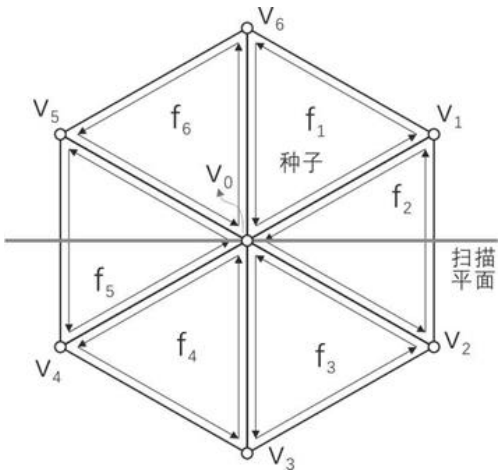
- **如果轮廓中包含的点的数量大于0，则将该轮廓添加至layer，否则直接放弃当前轮廓。**



基于拓扑模型的切片轮廓创建流程

4.拓扑模型切片

- **切片轮廓创建**
- **轮廓中点顶点数为0的异常情况　　——为什么会出现?**
  - **原因分析**
    - 如果当前截平面恰好过顶点v_0，而v_0被若干个面片共享（图中是6个），在这种情况下，面片f_1 ~ f_6都将被添加至扫描平面，即faces中。
    - 根据封装的findSeedFace函数，这些面片都有可能被选为种子面片，一旦某个面片被选为种子（如f_1），以上流程是能够进入第2层循环的，但因为面片和截平面没有交线，满足上述第1个退出条件，这样就得到了一条没有顶点的轮廓。
  - **这个问题看起来微不足道，而且很难被发现，但如果不去处理，在最坏的情况下，整个流程将被困在循环中跳不出来。**



轮廓包含点数为0异常

4.拓扑模型切片

- **切片轮廓创建**
- 将以上切片轮廓创建流程封装为函数**createLayerContours**，以补充TopoSlicer类。

TopoSlicer.py　TopoSlicer类定义，拓扑模型切片类。封装切片轮廓创建函数及剩余代码　7-7

```
1   ……                                          # 导入必要库，省略其他代码
2   class TopoSlicer:                           # 定义TopoSlicer类
3     ……                                        # 忽略类中其他代码
4     def createLayerContours(self, z, faces):  # 定义切片轮廓创建函数
5       layer = Layer(z)                        # 切片数据layer
6       for f in faces:                         # 清洗faces，置未使用
7         f.used = False
8       while True:                             # 第1层循环
9         f = self.findSeedFace(faces)          # 寻找种子面片 f
10        if f is None: break                   # 找不到种子，退出循环1
11        contour = Polyline()                  # 新建contour
12        while True:                           # 第2层循环
13          seg, edges, node = f.intersect(z)   # 计算面片和截平面交线
14          f.used = True                       # 设置面片为已用
15          if seg is None: break               # 退出条件1：无交线
16          contour.appendSegment(seg)          # 将交线添加至contour
17          if contour.isClosed(): break        # 退出条件2：轮廓封闭
18          f = self.findNextFace(edges, node)  # 寻找下一面片
19          if f is None: break                 # 退出条件3：无邻面
20        if contour.count() > 0:               # 判断轮廓顶点数是否大于0
21          layer.contours.append(contour)      # 大于0则将其添加至 layer
22      return layer
```

……

4.拓扑模型切片

- **切片轮廓创建**
- **createLayerContour**函数中，对得到的截交线段，通过Polyline类的appendSegment函数将其添加到当前轮廓中。
  - 在每一步循环中，尽管得到的截交线段必然是可以被直接添加到当前轮廓上的，但是不太确定的是，它到底是和当前轮廓的头部还是尾部相接。
  - 从理论上分析，对于一条新建的空轮廓，在对其添加第1条截交线段时可以不在意它的方向，但在添加第2条线段时，就要考虑一下它的走向。
  - 如果第2条线段是被拼接在了轮廓尾部，那么由追踪截交的原理可知，以后每条线段都将被拼接在轮廓尾部；而如果第2条线段被拼接在了轮廓头部，那么以后每条线段也都将被拼接在轮廓头部。
  - 最好不要将线段拼接到轮廓头部的，因为对于大都编程语言的动态数组来说，在数组头部插入一个数据的效率是低于在尾部添加一个数据的。
  - 以上代码选择了一种偷懒的方式，直接调用了appendSegment函数，让这个函数去判断到底应该将线段拼接到轮廓的头部或尾部。

——算法优化空间

伏笔

为了让截交线段能尽量被拼接到轮廓尾部，在代码7-6的**findNextFace**函数中，对于截交线段两交点都落在面片半边的情况，有意先检查半边e_1的邻面（第20行代码），后检查e_0的邻面，这是因为截交计算返回的线段是从e_0指向e_1的（参考TFace的intersect函数），先检查半边e_1显然是一种顺着轮廓生长方向的策略。

4.拓扑模型切片

- **切片轮廓创建**
- 基于createLayerContours函数，下面给出**TopoSlicer**类核心**slice**函数实现：

7-7续

```
23      def slice(self):                                    # 定义slice函数
24          self.topoModel.faces.sort(key=lambda t: t.zMin())   # 对面片最低点从低到高排序
25          zs = self.genLayerHeights()                     # 生成层高列表 zs
26          k = 0                                           # 扫描平面法
27          sweep = SweepPlane()                            # 定义扫描平面 sweep
28          for z in zs:
29              for i in range(len(sweep.triangles) - 1, -1, -1):   # 从sweep上移除不相交面片
30                  if z > sweep.triangles[i].zMax():
31                      del sweep.triangles[i]
32              for i in range(k, len(self.topoModel.faces)):       # 往sweep上添加新面片
33                  face = self.topoModel.faces[i]
34                  if z >= face.zMin() and z <= face.zMax():
35                      sweep.triangles.append(face)
36                  elif face.zMin() > z:
37                      k = i
38                      break
39              layer = self.createLayerContours(z, sweep.triangles)       # 生成切片轮廓
40              adjustPolygonDirs(layer.contours)           # 调整轮廓方向，外逆内顺
41              self.layers.append(layer)                   # 将数据添加至 layers 属性
42      def genLayerHeights(self):                          # 定义层高列表生成函数
43          xMin, xMax, yMin, yMax, zMin, zMax = self.stlModel.getBounds()
44          zs, z = [], zMin + self.layerThk
45          while z < zMax:
46              zs.append(z)
47              z += self.layerThk
48          return zs
```

扫描平面寻找当前层面片

生成层高列表

4.拓扑模型切片

- **切片轮廓创建：全局接口函数**

- 在和切片计算相关的全局函数模块SliceAlgo中添加一个TopoSlicer类的接口函数**slice_topo**，它的输入为常规STL模型，输出为包含所有切片轮廓信息的切片列表：

SliceAlgo.py	slice_topo：添加基于拓扑模型的切片算法	7-8

```
1  from TopoSlicer import *                    # 导入TopoSlicer模块
2  ……                                          # 忽略SliceAlgo其他导入模块及代码
3  def slice_topo(stlModel, layerThk):                  # 定义函数
4      return TopoSlicer(stlModel, layerThk).layers     # 返回切片列表
```

5.算法测试比较

- **比较对象slice_combine函数**
- 共封装了3种模型截交算法和3种截交线段拼接算法：

模型截交算法	截交线段拼接算法
暴力截交法	暴力拼接法
扫描平面截交法（sweep）	字典序拼接法(dorder)
层高匹配截交法(match)	字典查询拼接法(dlook)

- **优中选优**，将sweep和dlook两种算法组合，得到**slice_combine**函数，和slice_topo比较。

SliceAlgo.py | **slice_combine：添加扫描平面截交、字典查询拼接组合切片算法** | **7-9**

```
1  from TopoSlicer import *                    # 导入TopoSlicer模块
2  ……                                          # 忽略SliceAlgo其他导入模块及代码
3  def slice_combine(stlModel, layerThk):      # 定义组合切片函数
4      layers = intersectStl_sweep(stlModel, layerThk)   # 首先扫描平面截交
5      for layer in layers:
6          layer.contours = linkSegs_dlook(layer.segments)  # 然后字典查询拼接
7          layer.segments.clear()              # 清除layer中的线段，重要
8          adjustPolygonDirs(layer.contours)   # 调整轮廓方向，外逆内顺
9      return layers
```

在每层的线段被拼接后，**须清空该层上的所有截交线段**。这样做的原因是为了节省计算机内存空间，因为线段被拼接后，再在内存中保留这些线段已经没有意义。如果不删除这些线段，对于那些大型模型，在切片层数较大时，运行slice_combine函数很有可能导致内存溢出。（可以尝试下）

5.算法测试比较

- **测试代码**

- 测试主机CPU为intel Core i7-8700 @ 3.20GH。测试模型为bunny和vase模型，包含的面片个数分别约为27万和119万。对这两个模型，分别设置了1.0mm、0.5mm、0.2mm、0.1mm以及0.05mm这5种切片厚度。测试代码如下：

Test_topoSlicer.py	测试比较拓扑切片和组合切片两种方法效率	7-10

```
1   from SliceAlgo import *                          # 导入SliceAlgo库
2   src = vtk.vtkSTLReader()                         # 通过VTK库加载STL模型
3   src.SetFileName("E:\\STL\\bunny.stl")
4   stlModel = StlModel()
5   stlModel.extractFromVtkStlReader(src)
6   layerThks = [1.0, 0.5, 0.2, 0.1, 0.05]           # 切片厚度列表，5种切片厚度
7   for layerThk in layerThks:                       # 循环，一次性完成2种方法、5种层高切片
8       print('layerThk:', layerThk)                 # 打印当前层高
9       start = time.clock()
10      layers1 = slice_topo(stlModel, layerThk)     # 对拓扑切片计时
11      end = time.clock()
12      print("slice_topo: %f CPU seconds" % (end - start))
13      start = time.clock()
14      layers2 = slice_combine(stlModel, layerThk)  # 对组合切片计时
15      end = time.clock()
16      print("slice_combine: %f CPU seconds" % (end - start))
```

5.算法测试比较

- **测试结果**

拓扑切片和组合切片算法性能比较

模型	面片数	模型高度 (mm)	切片厚度 (mm)	层数	拓扑切片topo (秒)	组合切片combine (秒)	比值T/C	拓扑模型构建（秒）
bunny	270021	49.5	1.0	49	8.3	2.8	2.96	6.3
			0.5	98	8.9	5.0	1.78	5.4
			0.2	247	10.9	10.2	1.07	5.4
			0.1	494	15.3	18.4	0.83	5.4
			0.05	989	24.6	34.2	0.72	5.0
vase	1187016	381.4	1	384	41.0	29.6	1.39	22.9
			0.5	769	57.2	56.4	1.01	24.9
			0.2	1924	98.7	134.6	0.73	25.4
			0.1	3848	166.5	267.8	0.62	19.2
			0.05	7696	308.0	519.5	0.59	21.9

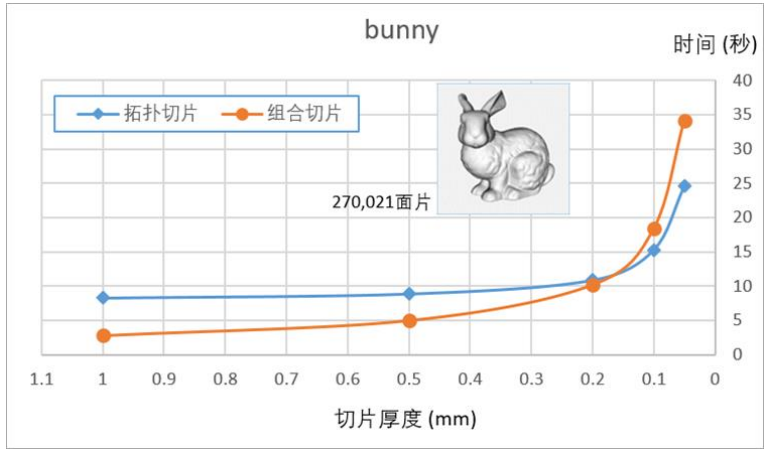
趋势:

- 切片厚度较大，即切片层数较少时，组合切片仍然优于拓扑切片，但它们的比值在缩小。
- 当切片厚度逐渐减小，相应的切片层数逐渐增多，切片计算量逐渐变大的情况下，组合切片算法的优势越来越小，甚至被拓扑切片反超。

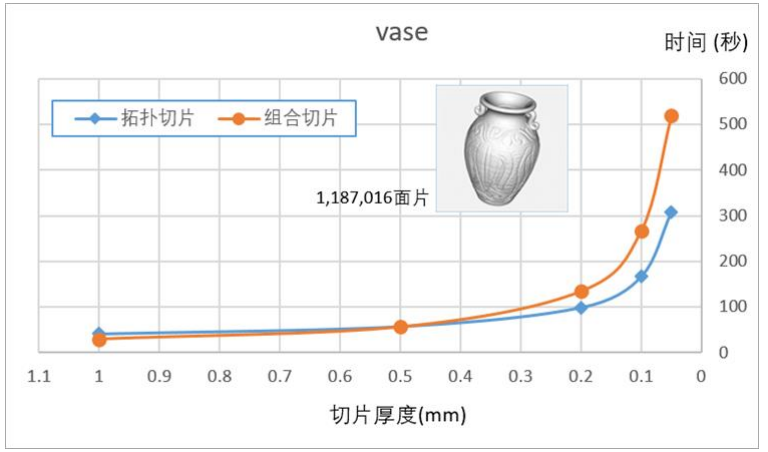
此外，对bunny和vase两个模型，**构建拓扑模型**的平均时间分别约为5.4秒和22秒。当切片层数较小时，构建模型的时间确实占据了整个拓扑切片的大部分时间。

5.算法测试比较

- **测试结果**



针对bunny模型两种切片方法执行时间趋势



针对vase模型两种切片方法执行时间趋势

结论:

- 两种切片方法在两个模型上的切片时间趋势是一样的，都是在切片厚度较大、切片层数不多时，组合切片优于拓扑切片；
- 但当切片厚度逐渐变小、切片层数逐渐增多时，拓扑切片优势逐渐体现并超越组合切片。

本章小结

- **这一章另辟蹊径，介绍并实现了基于拓扑模型的切片轮廓计算方法，它的功能相当于第五章介绍的截交算法和第六章介绍的拼接算法的总和，能够一步到位地从STL模型获取切片轮廓数据。**
  - 首先介绍了立体几何中顶点、棱边以及面的拓扑关系表示方法，包括从顶点索引到面，从面索引到顶点和半边，从半边索引到顶点、面以及对边。
  - 在此基础上，运用字典结构对输入的STL模型进行了拓扑重建，同时消除了模型中的冗余顶点。
  - 对得到的拓扑模型，首先使用扫描平面框架快速获取指定层高上和截平面相交的所有三角面片。然后我们从中挑选一个种子面片，并从这个种子面片出发，根据模型中点线面的拓扑关系，不断追踪、截交目标邻面，最终得到封闭轮廓。
  - 最后，对本章介绍的拓扑切片方法和基于第五、六章的组合切片方法进行了测试比较，结果表明：模型中面片数量越多、切面层厚越小，拓扑切片的优势越明显。





思考题

1. 在STL模型中，为什么说同一个顶点被重复保存了6次?
2. 尝试设计一种新的面片数据存储格式，以减小面片模型存储容量，同时能够保存模型拓扑信息。
3. 在棱边的拓扑结构中，为什么将一条棱边拆分成两条半边?
4. 在TFace类中，为什么需要重新实现一遍平面和三角面片的求交函数?
5. 至少用两种方法，快速去除STL模型上的冗余顶点。
6. 设计一种算法，去除平面多边形上的重复顶点。
7. 如何寻找并建立两条来自不同三角形的对边索引?
8. 设计一种算法，使用点、线、面之间的拓扑关系检测并修复STL模型上缝隙等缺陷。
9. 简述扫描平面法在拓扑切片中的作用。
10. 设计一种算法，生成具有均匀壁厚的轻量化抽壳模型。